RESOLUTION DES TRAVAUX PRATIQUES EC1

Coffre chiffre, infrastructure de gestion de cles,
canal TLS et cycle de vie des certificats

Module - Ingenierie de la cryptographie
Perimetre - TP 1, TP 2 et TP 3
Approche - implementation, justification, controles et limites d'execution

Note de lecture. Les consignes du sujet sont traitees comme des exigences de TP; les explications et reponses ci-dessous sont la resolution. Les commandes OpenSSL et Nginx sont fournies pour un laboratoire Linux. Les tests Python ont ete executables dans l'environnement de travail.

1. TP 1 - Chiffrement authentifie et derivation de cles

1.1 Format CFR1 et derivation

Le format est versionne par l'identifiant CFR1. Les trois parametres Argon2id sont stockes dans l'en-tete afin que chaque fichier puisse etre relu avec ses parametres d'origine. Les entiers sont codes en big-endian sur quatre octets.

Champ	Offsets	Valeur et usage
Identifiant	0-3	CFR1; selection du format.
t, m, p	4-15	Trois entiers 32 bits : temps, memoire en Kio, parallelisme.
Sel	16-31	16 octets aleatoires propres au fichier.
Nonce	32-43	12 octets aleatoires pour AES-GCM.
Chiffre + tag	44 et suite	Donnees chiffrees suivies du tag GCM de 16 octets.

Derivation. Argon2id transforme le mot de passe et le sel en une cle maitresse de 32 octets. Trois mesures medianes sur cet environnement donnent 0,220 s pour t=3, m=65536 Kio, p=4, puis 0,595 s pour t=8, m=131072 Kio, p=4. Ces derniers parametres sont retenus pour viser environ 0,5 seconde; le temps doit etre re-mesure sur la machine cible. HKDF-SHA256 derive la cle AES avec info = coffre-v1|fichier. Une etiquette de contexte separe les usages.

Chiffrement. AES-GCM chiffre les donnees et authentifie le bloc de 44 octets avec le nom UTF-8 du fichier comme donnees associees. Le nom, l'en-tete, le nonce, le chiffre et son tag doivent tous rester coherents pour que le dechiffrement reussisse.

Bornes de ressources. L'en-tete n'est pas authentifie avant la derivation. Le parseur borne donc t, m et p avant d'appeler Argon2id, afin qu'un en-tete hostile ne puisse pas demander une allocation ou un temps sans limite. Toute evolution au-dela de ces plafonds exige une politique/version de format explicite.

1.2 Implementation et tests de securite

coffre.py implemente cle_maitresse, sous_cle, chiffrer, dechiffrer et rechiffrer. Chaque appel de chiffrement tire un nouveau sel et un nouveau nonce via os.urandom. L'API AESGCM renvoie le chiffre avec son tag; aucune primitive n'est reimplementee.

Test	Resultat attendu	Propriete
Aller-retour	Le clair est retrouve octet pour octet.	Correction
Deux chiffrements	Deux blobs differents.	Sel et nonce frais
Mauvais mot de passe	InvalidTag.	Authentification de la cle
Octet du chiffre modifie	InvalidTag.	Integrite
Nom modifie	InvalidTag.	Liaison au contexte AAD
Rechiffrement	Nouveau mot de passe et parametres lisibles.	Migration

1.3 Questions de reflexion

1. Pourquoi le sel est-il public? Le sel n'est pas un secret. Il rend la cle derivee propre a ce fichier et empeche de reutiliser des tables precalculees ou de detecter directement deux memes mots de passe. Le publier n'affaiblit pas Argon2id; la protection vient du mot de passe, du cout de derivation et de la taille du sel.

2. Pourquoi le nonce aleatoire ne se repete-t-il pas sous une meme cle? Chaque fichier obtient un sel different et donc, sauf collision de sel, une cle AES differente. La regle GCM est l'unicite du nonce par cle, pas l'unicite globale entre toutes les cles. Avec un million de sels uniformes sur 128 bits, la probabilite de collision est de l'ordre de 10^-27. Pour un meme fichier, une nouvelle encryption produit aussi un nouveau sel et nonce.

3. Que risque-t-on sans le nom en donnees associees? Un attaquant pourrait deplacer ou renommer un blob valide vers un autre emplacement. L'application pourrait alors traiter son clair dans le mauvais contexte (par exemple, charger un fichier de configuration sous un nom reserve aux autorisations) sans que l'authentification detecte le changement.

4. Pourquoi authentifier l'en-tete? Sans l'en-tete dans AAD, un attaquant pourrait modifier la version ou les parametres Argon2id sans que le tag lie ces choix au contenu. Il pourrait rendre le fichier illisible, exploiter des parametres faibles acceptes ou provoquer un cout de calcul excessif. Les bornes de ressources restent necessaires car elles sont verifiees avant le tag.

5. Pourquoi une erreur generique? Une difference visible entre mauvais mot de passe, tag invalide, format corrompu ou parametres invalides pourrait servir d'oracle et donner des indices a un attaquant. L'interface doit renvoyer un seul message de rejet; les details techniques peuvent rester dans des journaux internes proteges.

1.4 Evolution des parametres

L'en-tete conserve les parametres ayant servi au fichier; dechiffrer lit donc les anciens parametres, tandis qu'un nouveau chiffrement utilise les valeurs courantes. rechiffrer ouvre le blob avec l'ancien mot de passe, puis cree un nouveau blob avec le nouveau mot de passe, un nouveau sel, un nouveau nonce et des parametres plus forts. Une evolution structurelle ou un changement d'algorithme utiliserait un nouvel identifiant, par exemple CFR2, sans supprimer le lecteur CFR1.

Resultat des tests. Six tests automatises passent : les cinq proprietes demandees et le rechiffrement avec renforcement des parametres. L'implementation propage InvalidTag au niveau bas; l'application appelante doit uniformiser le message presente a l'utilisateur.

2. TP 2 - Construire une infrastructure de gestion de cles

2.1 Hierarchie et profils

Certificat	Cle / duree	Extensions et contrainte
Racine	EC P-384 / 10 ans	CA:TRUE, pathlen:1; keyCertSign, cRLSign; auto-signee et hors ligne.
Intermediaire	EC P-256 / 5 ans	CA:TRUE, pathlen:0; contraintes DNS et courriel limitees a tp.local.
Serveur www.tp.local	EC P-256 / 90 jours	CA:FALSE; digitalSignature; serverAuth; SAN DNS www.tp.local et tp.local.
Client Alice/Bob	EC P-256 / 90 jours	CA:FALSE; digitalSignature; clientAuth; SAN alice@tp.local ou bob@tp.local.

2.2 Role des extensions

basicConstraints distingue une AC des certificats terminaux et borne la profondeur de la chaine. pathlen:0 interdit a l'intermediaire d'emettre une sous-AC. keyUsage limite l'usage de la cle; seules les AC ont keyCertSign et cRLSign. extendedKeyUsage separe les certificats serveur (serverAuth) des clients (clientAuth). subjectAltName porte les noms verifies par les clients. subjectKeyIdentifier et authorityKeyIdentifier facilitent la construction de la chaine. Les extensions d'AC qui fixent la confiance sont critiques.

L'intermediaire declare des contraintes de nom critiques pour DNS tp.local et le domaine de courriel tp.local. Les noms permis incluent tp.local et ses sous-domaines pour DNS; les identites de client restent dans le domaine de laboratoire.

2.3 Commandes et responsabilites

```
openssl genpkey -algorithm EC -pkeyopt ec_paramgen_curve:P-384 -out racine.key
openssl req -x509 -new -key racine.key -sha384 -days 3650 -addext basicConstraints=critical,CA:TRUE,
pathlen:1 -addext keyUsage=critical,keyCertSign,cRLSign -out racine.crt
openssl ca -batch -config racine.cnf -extensions ext_inter -days 1825   -in inter/inter.csr -out inter/
inter.crt
```

pki/pki.sh reprend ces etapes avec des fichiers de configuration, une base CA, des numeros de serie, des cles d'autorite chiffrees et une saisie silencieuse des phrases secretes. Les cles privees et les CSR sont ignores par Git. Dans une vraie exploitation, la racine est generee et utilisee lors d'une ceremonie hors ligne; le script pedagogique ne remplace pas cette separation physique.

2.4 Emission, validation et revocation

Le demandeur genere sa cle P-256 et une CSR signee par cette cle. L'AC valide l'identite et choisit elle-meme l'extension autorisee; la configuration utilise copy_extensions=none pour ne pas laisser le demandeur injecter basicConstraints ou d'autres extensions sensibles. La cle privee ne quitte pas le serveur ou le client.

```
openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt
-verify_hostname www.tp.local -purpose sslserver inter/serveur.crt
openssl verify -CAfile racine/racine.crt -untrusted inter/inter.crt
-purpose sslclient inter/alice.crt
```

Le premier controle doit reussir; le meme certificat avec autre.example doit echouer au controle de nom. Le certificat client doit etre accepte pour sslclient. Le certificat de demonstration www.example.com est signe par l'intermediaire, mais la validation de la chaine doit echouer avec le message permitted subtree violation. L'emission n'applique pas necessairement les contraintes de la chaine; le validateur, lui, les applique.

Alice est revoquee avec la raison keyCompromise. L'intermediaire publie une CRL; la racine publie aussi la sienne. Les deux CRL sont concatenees dans crls.pem, et openssl verify -crl_check_all doit marquer Alice comme revoquee. Bob recoit ensuite un certificat neuf; le certificat d'Alice ne doit plus etre accepte.

2.5 Questions de reflexion

1. Compromission. La racine reste hors ligne pour reduire l'exposition. Si l'intermediaire est compromis, on suspend son emission, on le revoque avec la racine, on publie la nouvelle CRL, on cree un intermediaire de remplacement et on reemet les feuilles. Si la racine est compromise, la chaine de confiance est perdue : il faut creer une nouvelle racine, la distribuer hors bande, retirer l'ancienne des magasins de confiance et reemettre toute la hierarchie.

2. pathlen:0. Il autorise l'intermediaire a signer des feuilles mais pas d'autres AC. Sans cette limite, une AC compromise pourrait deleguer l'emission et elargir le nombre d'acteurs capables de creer des certificats.

3. Cle du serveur. Le serveur cree sa cle privee et envoie seulement une CSR. La cle ne traverse pas le reseau et l'AC ne la connait jamais; cela reduit les copies et les risques de fuite.

4. CN et SAN. Les clients modernes comparent le nom d'hote avec le SAN. Le CN est une information de sujet et ne remplace pas un SAN correct.

5. CRL, OCSP, duree courte. Une CRL peut etre volumineuse et son contenu devient stale entre deux publications. OCSP donne un etat plus recent, mais ajoute latence, dependance au repondeur et fuite d'information sur les sites visites; certains clients ont aussi pratique l'echec ouvert. Des certificats courts reduisent la fenetre d'exposition, au prix d'un renouvellement automatise et surveille.

2.6 Politique minimale de certification

Regle	Politique proposee
1. Eligibilite	Seuls les administrateurs et services inscrits a l'inventaire peuvent demander un certificat.
2. Identite	Un operateur autorise valide le compte; un controle DNS ou une approbation de proprietaire confirme les noms.
3. Cle	Le demandeur genere une cle EC P-256 localement; aucune cle privee n'est envoyee a l'AC.
4. CSR	La CSR doit prouver la possession de la cle et correspondre au sujet demande.
5. Emission	L'intermediaire emet seulement les profils serveur ou client approuves et journalise serie, sujet, SAN et operateur.
6. Duree	Feuilles valides 90 jours; renouvellement automatise avant 30 jours restants.
7. Protection	La racine chiffree est conservee hors ligne; l'intermediaire est protegee par controle d'acces et phrase secrete hors depot.
8. Revocation	Une compromission est signalee immediatement; le certificat est revoque et une CRL est publiee.
9. Distribution	Les clients font confiance uniquement a la racine de laboratoire; les serveurs envoient feuille et intermediaire, jamais la racine.
10. Audit	Les emissions, renouvellements, echecs et revocations sont journalises; l'inventaire et les dates sont controles quotidiennement.

Pourquoi l'emetteur et le sujet de la racine sont identiques? Le certificat racine est auto-signe. Cette signature protege le contenu du certificat mais ne cree pas la confiance : celle-ci provient de l'installation explicite de la racine dans le magasin de confiance.

3. TP 3 - Canal TLS et authentification mutuelle

3.1 Configuration Nginx

Directive	Effet et justification
ssl_certificate / ssl_certificate_key	Le serveur presente feuille + intermediaire et utilise sa cle privee locale.
ssl_protocols TLSv1.2 TLSv1.3	Refuse SSL et TLS 1.0/1.1.
ssl_ciphers	Pour TLS 1.2, limite le choix a ECDHE-ECDSA avec AES-GCM ou ChaCha20-Poly1305, donc PFS et AEAD.
ssl_session_tickets off	Desactive les tickets stateless tant que les cles de ticket ne sont pas gerees avec rotation.
ssl_client_certificate / ssl_verify_client on	Demande un certificat client et valide sa chaine vers la racine.
ssl_verify_depth 2 / ssl_crl	Borne la profondeur et verifie les CRL concatenees.
Strict-Transport-Security	Demande au navigateur d'utiliser HTTPS pendant un an apres reception de l'en-tete.
$ssl_client_s_dn	Affiche l'identite X.509 du client authentifie pour le service de laboratoire.

Les suites TLS 1.3 sont gerees par OpenSSL et ne sont pas regies par ssl_ciphers dans les versions courantes de Nginx. Elles doivent rester sur des suites AEAD modernes. La configuration limite ici les suites TLS 1.2 compatibles avec le certificat ECDSA.

3.2 Poignee de main et verification

Le client envoie ClientHello avec les versions, suites, alea et key_share. Le serveur repond ServerHello avec la suite et son key_share. Les deux parties derivent les cles ephemeres; le serveur envoie ensuite, chiffre, la chaine, CertificateVerify et Finished. CertificateVerify signe la transcription; Finished confirme les secrets derives.

En TLS 1.3, le certificat serveur n'est pas visible en clair sur le reseau : apres ServerHello, les messages suivants de la poignee de main sont chiffres sous les cles derivees. Wireshark peut les afficher si on lui fournit les secrets de session dans un fichier de key log. En TLS 1.2, le certificat est normalement visible.

```
openssl s_client -connect www.tp.local:443 -servername www.tp.local
-verify_hostname www.tp.local -CAfile racine/racine.crt
-cert bob.crt -key bob.key -cert_chain inter/inter.crt -trace
```

Sur la machine de labo, relever dans la sortie le protocole negocie, la suite et Verification: OK. Le groupe d'echange se releve dans les extensions key_share avec -trace. Sans certificat client, le serveur doit refuser la connexion; avec Alice, la validation doit echouer apres sa revocation. TLS 1.1 doit echouer.

3.3 Supervision et renouvellement

tls/surveiller.py parcourt recursivement les PEM/DER. Pour chaque certificat, il compare l'expiration a l'heure UTC : OK a partir de 30 jours, ALERTE entre 7 et 30 jours, CRITIQUE sous 7 jours ou apres expiration. Le code de sortie est le risque maximal : 0, 1 ou 2. Un certificat illisible ou un inventaire vide produit 2 pour eviter un faux etat sain.

Le test tls/test_surveiller.py fabrique temporairement un certificat valide cinq jours avec une cle gardee en memoire. Le script de surveillance le classe CRITIQUE et renvoie 2; aucun secret n'est ecrit dans le depot.

tls/renouveler.sh quitte sans changement si le certificat a encore plus de 30 jours. Sinon, il genere une nouvelle cle et CSR sur le serveur, fait signer la CSR, valide la chaine et la correspondance cle/certificat, sauvegarde les anciens fichiers, installe les nouveaux, execute nginx -t puis recharge Nginx. Une erreur pendant le deploiement restaure les fichiers precedents et tente un nouveau controle/rechargement.

3.4 Questions de reflexion

1. Pourquoi ne pas envoyer la racine? Le client doit deja faire confiance a la racine par son magasin local. La transmettre n'ajoute pas cette confiance; le serveur envoie le certificat feuille et les intermediaires necessaires a la construction du chemin.

2. Tickets de session. Les tickets sont chiffres avec une cle de serveur. Si cette cle n'est pas renouvelee, un attaquant qui la recupere peut conserver une capacite de dechiffrement ou de reprise plus longtemps. Les desactiver evite ce risque de cle longue duree; si on les active, la cle de ticket doit etre geree et tournee.

3. HSTS. Apres reception, le navigateur remplace les acces HTTP par HTTPS pendant max-age et refuse les contournements de certificat. Cela protege contre le downgrade et le SSL stripping pour les visites suivantes; le premier contact n'est protege que si le domaine est precharge. Cette configuration ne couvre pas les sous-domaines car includeSubDomains n'est pas present.

4. Durees publiques. Selon le calendrier donne dans le cours, la limite passe a 100 jours en 2027 puis 47 jours en 2029. Une organisation qui renouvelle a la main augmente fortement le risque d'oubli et d'indisponibilite; inventaire, alertes, renouvellement automatise et retour arriere deviennent necessaires.

5. Chaine incomplete. Certains clients possedent deja l'intermediaire en cache ou savent le recuperer via les informations d'acces a l'autorite; d'autres n'ont ni cache ni acces et echouent. Le serveur doit envoyer la feuille et les intermediaires, pas dependre de l'etat du client.

6. Fuite de cle serveur. Isoler le serveur et informer l'AC; revoquer le certificat; generer une nouvelle paire sur une machine saine; emettre et verifier un nouveau certificat; deployer la chaine sans interruption; publier/rafraichir les CRL; invalider les sessions et tickets concernes; rechercher les traces d'exploitation et informer les parties concernees.

4. Execution, limites et remise

4.1 Controles realises dans cet environnement

Element	Etat
Tests du coffre	6 tests pytest passent : aller-retour, alea, mauvais secret, modification, renommage et migration.
Surveillance 5 jours	Test execute : le certificat est classe CRITIQUE et le code de sortie vaut 2.
Syntaxe Python	Les modules et scripts Python compilent.
Syntaxe Bash	pki.sh et renouveler.sh passent le controle bash -n.
PKI avec OpenSSL 3.2.4	pki.sh execute sur copie temporaire : racine/intermediaire, SAN serveur, contraintes, CRL; www.example.com est rejete et Alice revoquee tandis que Bob est valide.
Nginx / handshake TLS	Nginx n'est pas installe ici; la configuration, mTLS, TLS 1.1 et la suite/groupe negocies restent a relever sur le serveur Linux.

La chaine PKI, les usages, le mauvais nom, la violation de nameConstraints et la CRL ont ete verifies avec OpenSSL 3.2.4 dans un repertoire temporaire ensuite nettoye. Les resultats de suite TLS, de groupe ECDHE, de verification Nginx et les captures restent dependants de l'execution du serveur Linux. Il faut y reporter la sortie reelle avant de presenter une capture ou une mesure comme observation.

4.2 Regles de securite appliquees au depot

Le depot contient le code, les configurations, le rapport et des scripts de generation; il ne contient aucun mot de passe, aucune cle privee, aucune CSR ni base d'emission. Les cles de laboratoire generees localement sont ignorees par Git. Les secrets de renouvellement doivent venir d'un stockage protege, jamais d'une ligne de commande contenant la phrase secrete ni d'un fichier versionne.

Sources de travail

Le raisonnement suit le cours fourni : derivation de cles et Argon2id (chapitre 2), AEAD/GCM (chapitre 3), certificats X.509 et validation de chaine (chapitre 5), TLS 1.3 et confidentialite persistante (chapitre 6). Les exigences operationnelles viennent du sujet TP EC1, pages 3 a 10. Les observations de laboratoire doivent etre completees avec les sorties reelles sur Linux.

Fin du compte rendu